



OPEN Precise identification of tomato leaf diseases based on the real background of DPEN

Wanlin Gao^{1,3}, Xin Zhang², Yuxin Tian² & Yinxue Shi^{2,3}✉

As an important economic crop, tomato is vulnerable to various diseases, and these diseases often have high visual similarity, making identification difficult. A delay in diagnosis can have a significant effect on tomato yields. Traditional manual visual inspection methods have poor accuracy, while laboratory diagnostic methods are inefficient, making them unsuitable for large-scale agricultural scenarios. To address this challenge, this study involved the collection and construction of a tomato leaf disease dataset in a real planting environment, and AutoAugment was used to achieve sample diversity and balance the number of training samples in different categories. Additionally, a dual-path ensemble network (DPEN) was proposed, which combines the multiscale feature extraction advantages of GoogLeNet with the dense connection mechanism of DenseNet121. The experimental results show that, compared with the comparison models, the DPEN achieves an identification precision of 98.80% on the self-built dataset, which is an improvement of 2.33% to 9.24%, and a reduction in the number of parameters by 7.09 M compared with GoogLeNet and 2.07 M compared with DenseNet121. The experimental results on public datasets further demonstrated the accuracy of the proposed DPEN model in identifying tomato leaf diseases in complex backgrounds. These results prove that the DPEN model can achieve precise, rapid, and efficient identification of tomato leaf diseases in complex backgrounds, providing technical support for smart agriculture applications.

Research background and significance

The agricultural sector is confronted with numerous challenges, among which crop diseases stand out prominently. According to statistics, crop yield losses due to diseases worldwide amount to 40% to 50%¹. Tomato, as one of the top three vegetable trade products globally, holds a significant position in the global vegetable trade. However, tomato plants are prone to various diseases, which occur mainly on their leaves. Once infected, these diseases can rapidly spread to healthy plants within a short period, posing a serious threat to tomato yield and quality². Traditional disease detection methods, such as visual inspection or chemical analysis of diseased leaf areas, are subjective and inefficient and fail to meet the demands of actual agricultural production³. Therefore, designing a rapid, automatic, and precise method for identifying tomato leaf diseases is highly important⁴.

In recent years, artificial intelligence technology has made remarkable progress in image recognition, object detection, and image segmentation. In particular, models based on convolutional neural networks (CNNs) have demonstrated outstanding performance in image recognition.

A CNN is a network structure capable of autonomously learning features from input images⁵. A basic CNN typically consists of convolutional layers, activation functions, pooling layers, and fully connected layers⁶. The convolutional layer is the key to building a CNN, whose main function is to extract data features. Activation functions are usually applied after the convolutional layer to add nonlinearity and help the model learn more complex patterns. Pooling layers are used to downsample feature maps, reducing the computational load and the number of parameters while enhancing feature invariance. Fully connected layers are usually located at the tail of the CNN. The main purpose of these methods is to achieve classification⁷. During training, the CNN uses the backpropagation algorithm to update weights and minimize the loss function value to optimize model performance.

The development of CNNs can be traced back to 1998. The LeNet model designed by LeCun et al.⁸ provided a solid foundation for the development of modern CNNs. With continuous technological advancements, Krizhevsky et al.⁹ proposed the deep AlexNet model structure and won the championship in the ImageNet

¹International College of South and Southeast Asia, Pu'er University, Pu'er 665000, China. ²College of Information and Electrical Engineering, China Agricultural University, Beijing 100083, China. ³Key Laboratory of Agricultural Informatization Standardization, Ministry of Agriculture and Rural Affairs, Beijing 100083, China. ✉email: syx2821@cau.edu.cn

competition that year. This achievement greatly promoted the research boom of deep CNNs. Subsequently, Simonyan et al.¹⁰ proposed VGGNet, which uses smaller 3×3 convolution kernels stacked to replace larger kernels, thereby achieving better model performance while reducing the number of parameters. In the same year, Szegedy et al.¹¹ proposed GoogLeNet, which reduces the number of parameters while introducing the inception block to increase the model width. In 2016, He et al.¹² proposed ResNet, which solved the vanishing gradient problem in deep CNNs by introducing shortcut connections in VGGNet. In 2017, Huang et al.¹³ proposed DenseNet, which achieves better performance and fewer parameters from the perspective of feature reuse. These landmark achievements not only demonstrated the strong potential of deep CNNs in the field of images but also provided valuable references and inspirations for subsequent research.

With the development of computer technology, plant disease detection methods based on deep learning models have been widely applied in the agricultural sector¹⁴. Research on the use of CNNs to automatically identify tomato leaf diseases has also emerged. By applying these technologies, we can provide more efficient technical support and achieve rapid and accurate identification of tomato leaf diseases.

Research status

At present, research on plant leaf disease recognition can be divided into two directions: leaf disease recognition in simple backgrounds and that in complex backgrounds.

In studies using plant leaf data with simple backgrounds, the background of such data is single and has less noise, so the deep learning models trained with this type of data usually achieve high recognition accuracy on the test set. Guo et al.¹⁵ proposed the ET3-MobileNetV3 model, which is based on MobileNetV3, to address the problems of weak generalization ability and the large number of parameters of existing convolutional neural network models for apple leaf disease recognition. Through improvements such as attention mechanisms, fully connected layers, and operators, combined with transfer learning, the model achieved an average accuracy of 95.62% on the test set of the self-built simple background apple leaf disease dataset, with a model weight file size of only 6.29 MB. Liu et al.¹⁶ proposed a deep convolutional neural network model for apple leaf disease recognition, which was based on the improvement of AlexNet. Compared with those of the standard AlexNet model, the model parameters were reduced by 51 M. Experiments on the self-built simple background apple leaf dataset revealed that the proposed model achieved an overall accuracy of 97.62% for four types of apple leaf diseases. Xu et al.¹⁷ designed an accurate recognition model for peanut leaf diseases via Xception via a partial-activated feature fusion module and two self-constrained attention-augmented branches. The experimental results revealed that the model achieved a recognition accuracy of 99.69% on the self-built simple background peanut leaf disease recognition task, which was 9.67% - 23.34% higher than that of InceptionV4, ResNet34, and MobileNetV3. Dhanalakshmi et al.¹⁸ proposed a tomato leaf disease recognition method based on Inception, and the constructed model combined the advantages of deep residual and dense networks. The experimental results on the PlantVillage simple background public dataset showed that the model achieved an average recognition accuracy of 96%.

In actual agricultural production, disease recognition often faces more complex background environments, making it difficult for models trained with simple background data to meet actual needs. Therefore, some researchers trained models using simple background plant leaf data and then applied the trained models to complex background plant leaf disease recognition tasks. However, the experimental results revealed relatively low recognition accuracy. Xu et al.¹⁹ proposed an apple leaf disease recognition model, CaTNet, which uses simple background images from PlantVillage, Plant Pathology, and AI Studio as the training set and self-built complex background data as the test set. The experimental results revealed that the proposed model achieved a recognition accuracy of 79.35% on complex background leaf disease data.

In studies using complex background plant leaf data, owing to the significant interference of the background of such leaf data, effectively extracting the disease features of complex background leaves is difficult. Therefore, the recognition accuracy of some proposed models still has room for improvement. Li et al.¹ proposed the S-DenseNet-E method for the rapid recognition of apple leaf diseases. First, a lightweight improvement based on a dense block was made to obtain S-DenseNet. Experiments on the Plant Pathology 2021-FGVC8 public dataset with complex backgrounds revealed that the F1-Score of S-DenseNet reached 85.14%, which was higher than that of commonly used CNN models. The recognition inference time was 33.03 ms, which was lower than that of MobileNetV2. To address the issue of a low F1-Score for unidentifiable diseases, an auxiliary model was added to S-DenseNet to specifically identify unidentifiable diseases, forming the S-DenseNet-E model. Experiments on the same dataset indicated that the F1-Score of S-DenseNet-E for unidentifiable diseases increased by 4.28%, and the recognition inference time increased by only 5.89 ms, enabling rapid and accurate identification of crop diseases. Hidayatulloh et al.²⁰ used the SqueezeNet model to identify tomato leaf diseases in complex backgrounds. The researchers conducted experiments using a self-built dataset of tomato leaf diseases in complex backgrounds. Under 5-fold cross-validation, the average recognition accuracy was 86.92%. Wang et al.²¹ proposed a parallel structure DEFL model based on EfficientNet-B0 and DenseNet121 for identifying apple leaf diseases. They used a focal loss function with a label smoothing strategy to increase the model's attention to difficult-to-identify categories. Experiments on the Plant Pathology 2021-FGVC8 public dataset with complex backgrounds revealed that the average recognition accuracy of DEFL was 98.47%. Huang et al.²² proposed a method of first segmentation and then recognition to solve the problem of poor generalization ability of complex background tomato leaf disease recognition. The segmentation model was based on the improved VGG16, which replaces the fully connected layer with a deconvolution layer to form the fully convolutional FCN model. The recognition model was based on the improved DPN to obtain the SNDPN model. The experimental results on a dataset composed of complex background data crawled from the internet and PlantVillage revealed that when the FCN was used for foreground segmentation and the SNDPN was used for disease recognition, the disease recognition accuracy was 97.59%. Liu et al.²³ designed a plant disease recognition model CLT that integrates a

Vision Transformer and a CNN. It uses convolutional layers to extract shallow local features and a transformer to learn global features, effectively solving the problem of difficulty in recognizing plant disease images in real scenarios. The experimental results on the PlantDoc complex background public dataset revealed that the proposed model achieved an average accuracy of 77.91% on the test set, which was 13.91% higher than that of ViT (Vision Transformer) and outperformed those of the Swin transformer, InceptionV3, and ResNet50. Zhao et al.⁷ proposed a dual transfer learning method to address the difficulty of feature extraction for leaf diseases in complex backgrounds. This method first improves the pretrained ResNet50 model via SENet to obtain SE-ResNet50 and then trains the model via the AI Challenger 2018 public dataset with simple backgrounds. Finally, the model was trained via a self-built complex background tomato leaf dataset. The results showed that the model achieved a recognition accuracy of 97.24% on the self-built complex background tomato leaf dataset. Omer et al.²⁴ improved the YOLOv5l model by using the bottleneck CSP module and the CBAM attention module. While maintaining high performance, the model's weight file was only 13.6 MB in size, significantly reducing storage and computational costs. The experimental results on a self-built complex background cucumber leaf image dataset revealed that the precision and recall rates reached 73.80% and 73.90%, respectively, and that the mPA reached 80.10%. Zhang et al.²⁵ introduced the attention module and sharp cosine convolution into the dense block and transition layer based on the improvement of DenseNet. They conducted transfer learning via the PlantVillage public dataset and retrained the model via a self-built complex background lotus leaf dataset. The experimental results on the self-built complex background lotus leaf dataset indicated that the proposed model achieved a recognition accuracy of 91.34%. Tewari et al.²⁶ proposed a method for cassava leaf disease recognition based on the improvement of MobileNetV2, which combines depthwise separable convolution and a spatial attention mechanism. The experimental results on a self-built complex background cassava leaf image dataset revealed that the validation accuracy reached 98%, and the test accuracy reached 75%. Finally, a mobile application was designed and developed for deploying the proposed model, effectively addressing the needs of farmers. Upadhyay et al.²⁷ employed a pretrained ResNet-50 to determine whether a leaf is infected by diseases. Subsequently, SegLearner was applied to perform semantic segmentation for extracting lesion regions, and the disease severity was quantified based on pixel-level statistics. Experiments conducted on Mask-LeafDataset and ADE20K reported mIoU scores of 91.4% and 45.8%, respectively, demonstrating promising generalization performance.

In summary, existing studies have demonstrated that plant leaf disease recognition methods based on convolutional neural networks and ViT have achieved remarkable progress. Particularly on simple-background datasets such as PlantVillage, these models can achieve nearly perfect classification accuracy, confirming the feasibility and great potential of deep learning techniques for intelligent agricultural disease diagnosis. However, there remains a considerable gap between such datasets and real agricultural environments. Although recent research has begun to address plant disease recognition under complex backgrounds, overall performance is still inadequate. Highly distracting background environments, varying illumination conditions, and strong visual similarities across different disease categories pose substantial challenges to accurate disease recognition, leading to unsatisfactory recognition performance for certain categories. Therefore, it is imperative to construct available datasets that realistically reflect field conditions and to develop deep learning models capable of accurate disease recognition under complex backgrounds, thereby promoting the practical deployment of automated plant disease recognition in real-world agricultural production.

Related work

To address the above problems, this study conducted multiple onsite data collections during the growth cycle of tomato plants in a greenhouse environment and created a complex background tomato leaf disease dataset. To address the poor model accuracy caused by the difficulty in effectively extracting disease features from complex background leaves, a DPEN model capable of accurately identifying tomato leaf diseases was designed and implemented, providing technical support for more efficient agricultural production activities. The main work of this paper is as follows:

- (1) To address the lack of large-scale high-quality public datasets of complex background tomato leaf diseases, this study conducted multiple onsite data collections during the growth cycle of tomato plants in a greenhouse environment and constructed a dataset of tomato leaf images in complex backgrounds, including 2558 images of four types of leaves: magnesium deficiency, healthy, virus disease with curled leaves, and leaf mold.
- (2) To address the problems of sample imbalance and lack of diversity in the training dataset, the AutoAugment data augmentation method was used to balance the sample size of the training dataset and increase the diversity of the training data, further improving the generalization ability of the model.
- (3) To address the difficulty in recognition caused by significant interference in complex background tomato leaf diseases, a complex background tomato leaf disease recognition method based on DPEN was constructed. This model was improved based on GoogLeNet and DenseNet121. The former is good at extracting multiscale features, whereas the latter focuses on feature reuse. By combining the advantages of both methods, the DPEN model was obtained, and the convergence speed and stability during the training stage were improved by introducing a branch classifier. Finally, experiments were conducted on the self-built dataset and public datasets, and the constructed DPEN model was evaluated via four evaluation metrics—accuracy, recall rate, F1-Score, and model parameter quantity—to prove the reliability and robustness of the model.

The rest of this paper is organized as follows. Section [Dataset](#) introduces the self-built dataset images and public datasets for tomato leaf disease detection. Section [Methodology](#) introduces the model structure of the tomato leaf disease recognition method. Section [Experimental analysis](#) introduces the experimental process and results and analyzes the actual effect of the model. Section [Conclusion](#) summarizes the research results of this paper and discusses the possible future research directions and application prospects of the model.

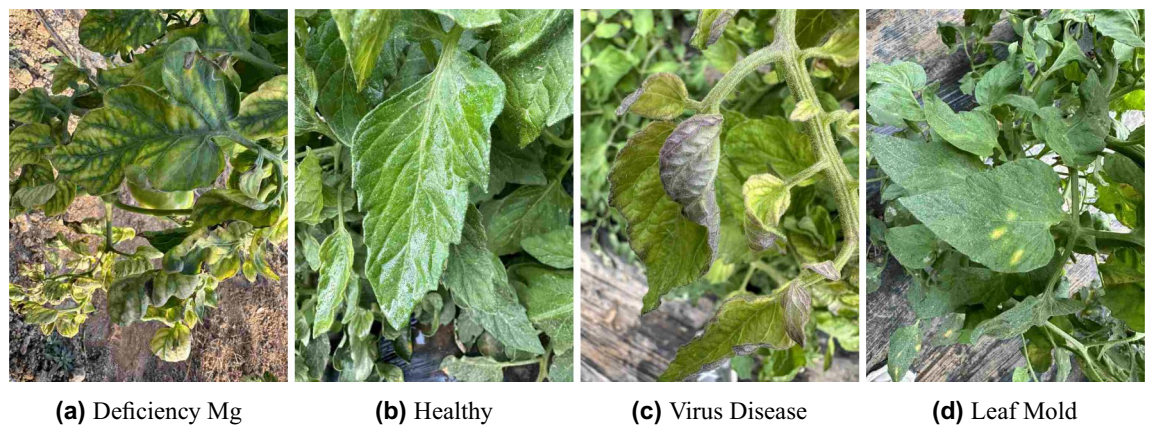


Fig. 1. Examples of self-built dataset images.

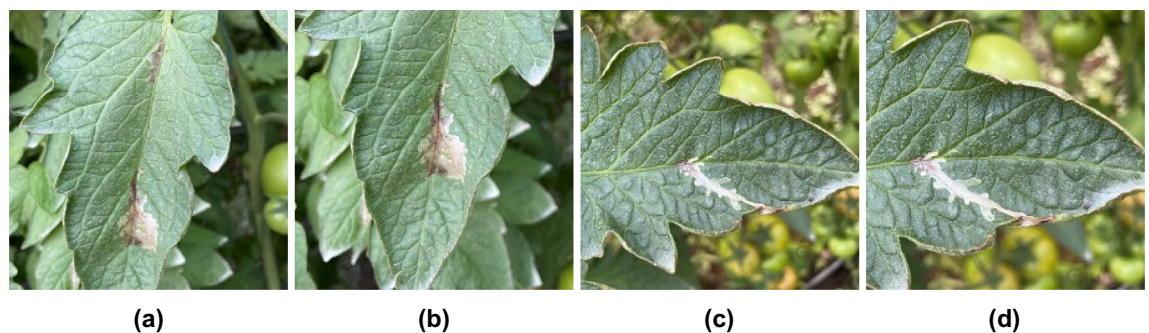


Fig. 2. Similar sample image example.

Dataset

Dataset introduction

This paper conducts experiments using both a self-built dataset and a public dataset. The self-built dataset is composed of multiple onsite collections collected throughout the growth cycle of tomato plants, providing a data source for the subsequent training of deep learning models. This dataset consists of tomato leaf images in complex backgrounds, specifically including 666 images of leaves with magnesium deficiency (Deficiency Mg), 608 images of healthy leaves (Healthy), 574 images of leaves with virus disease (Virus Disease), and 710 images of leaves with a leaf mold (Leaf Mold), totaling 2,558 images. The data are divided into a training set and a test set at an 8:2 ratio. All images were collected from the “Zan Jia Cai Yuan” farm in Yantai city, Shandong Province, using an iPhone 14 Pro to capture leaves from different tomato plants at various angles. The images were taken between 2 p.m. and 6 p.m. on sunny afternoon when the external light conditions were ideal, which contributed to the clarity and color representation of the images. The background of the collected images was either soil or tomato leaves, ensuring that the dataset has high practical application value. All images were obtained in automatic exposure mode and transferred to the computer in JPG format for processing. Examples of images from the self-built dataset are shown in Fig. 1.

The public dataset (www.kaggle.com/datasets/paulrosero/tomato-leaf-illness-detection) consists of 123 images of late blight leaves, 135 images of healthy leaves, 85 images of gray spot leaves, and 141 images of magnesium deficiency leaves, totaling 484 images. The samples for the training set and test set have already been divided at an 8:2 ratio. However, this division ratio makes the cost of prediction errors very high, meaning that the prediction error of a single sample sharply reduces the accuracy of the test set. Therefore, the predefined training set and test set were merged and redivided at a 7:3 ratio. During the data merging process, we found that some samples originated from the cropping of the original samples, as shown in Fig. 2. Thus, these data were separated into a training set to avoid having two highly similar samples in both the training set and the test set. Examples of images from the tomato-leaf-illness-detection public dataset are shown in Fig. 3.

Dataset augmentation

The sample sizes of the training sets for the four categories in the self-built dataset are imbalanced. Sample imbalance can lead the model to tend to learn the features of categories with more samples during the training process while ignoring those with fewer samples, thereby weakening its recognition ability and generalization ability for categories with fewer samples²⁸. The training set of the public dataset has too few samples, which can prevent the model from learning sufficient information and features from the data.

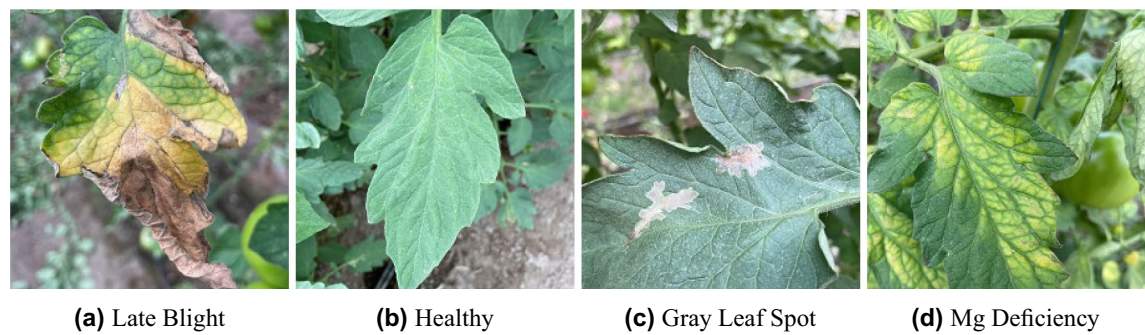


Fig. 3. Example images from the tomato leaf illness detection public dataset.

Self-built dataset			Public dataset		
Disease	Before	After	Disease	Before	After
Deficiency Mg	540	1000	Late Blight	86	1000
Healthy	492	1000	Healthy	94	1000
Virus Disease	462	1000	Gray Leaf Spot	59	1000
Leaf Mold	564	1000	Mg Deficiency	99	1000

Table 1. The quantity of data before and after training set data augmentation.

To solve the above problems and increase the diversity of training data, data augmentation techniques have been widely used in deep learning tasks. Image data augmentation is one of the most commonly used forms of augmentation, which generates more training samples by transforming the existing data. Through these transformations, the model can not only learn more diverse features to enhance the learnability of the data but also improve the recognition accuracy for the categories with fewer samples, thereby avoiding overfitting to the data of the categories with more samples. In this work, the AutoAugment data augmentation method is adopted to expand the training sets of both the self-built dataset and the public dataset to 1000 images per category. The changes in the training set data volume are shown in Table 1.

Data augmentation typically relies on fundamental image transformation techniques such as rotation, translation, scaling, cropping, and flipping. However, these methods often apply a single transformation to the original image and utilize manually specified hyperparameters, which limits their flexibility. Moreover, since the generated data may be highly similar to the original samples, models tend to learn these transformations rather than discriminative features across different images, potentially leading to overfitting. To address this issue, AutoAugment²⁹ proposes an automated framework for searching optimal data augmentation strategies.

In this study, data augmentation is performed using the ImageNetPolicy from AutoAugment. The core principle of AutoAugment is to leverage reinforcement learning (RL) to automatically explore optimal augmentation strategies, thereby overcoming the limitations associated with manually selecting operation types and hyperparameters. AutoAugment predefines a set of policies for four public datasets—CIFAR-10, CIFAR-100, ImageNet, and SVHN—where each dataset is associated with 25 sub-policies. Each sub-policy consists of two image augmentation operations, with each operation characterized by an application probability and an augmentation magnitude. The supported operations include shearX, shearY, translateX, translateY, rotate, color adjustment, posterize, solarize, contrast adjustment, sharpness adjustment, brightness adjustment, autocontrast, equalize, and invert. During training, a sub-policy is randomly selected for each mini-batch, and each operation within the sub-policy is applied according to its predefined probability. For example, a sub-policy in ImageNetPolicy might first posterize a graph image with magnitude 8 with probability 0.4, and then rotate the image with magnitude 9 with probability 0.6. These operations are implemented via dedicated functions, with a fill color specified to handle blank regions introduced by geometric transformations. The RL controller adjusts the policies based on validation set accuracy; operations with poor performance are either assigned lower application probabilities or excluded entirely, ultimately forming a set of 25 effective sub-policies. Experimental results on CIFAR-10, CIFAR-100, ImageNet, and SVHN demonstrate that this approach significantly improves model performance. Moreover, AutoAugment experiments indicate that policies learned on large-scale datasets exhibit strong transferability. For instance, strategies learned on ImageNet can be directly applied to fine-grained classification datasets without additional fine-tuning. Examples of augmented data from the self-constructed dataset are shown in Fig. 4.

Methodology
The structure of the DPEN

The DPEN model proposed in this study has a parallel structure, consisting of the MyGoogLeNet branch and the MyDenseNet121 branch. It integrates the advantages of both DenseNet and GoogLeNet and introduces the

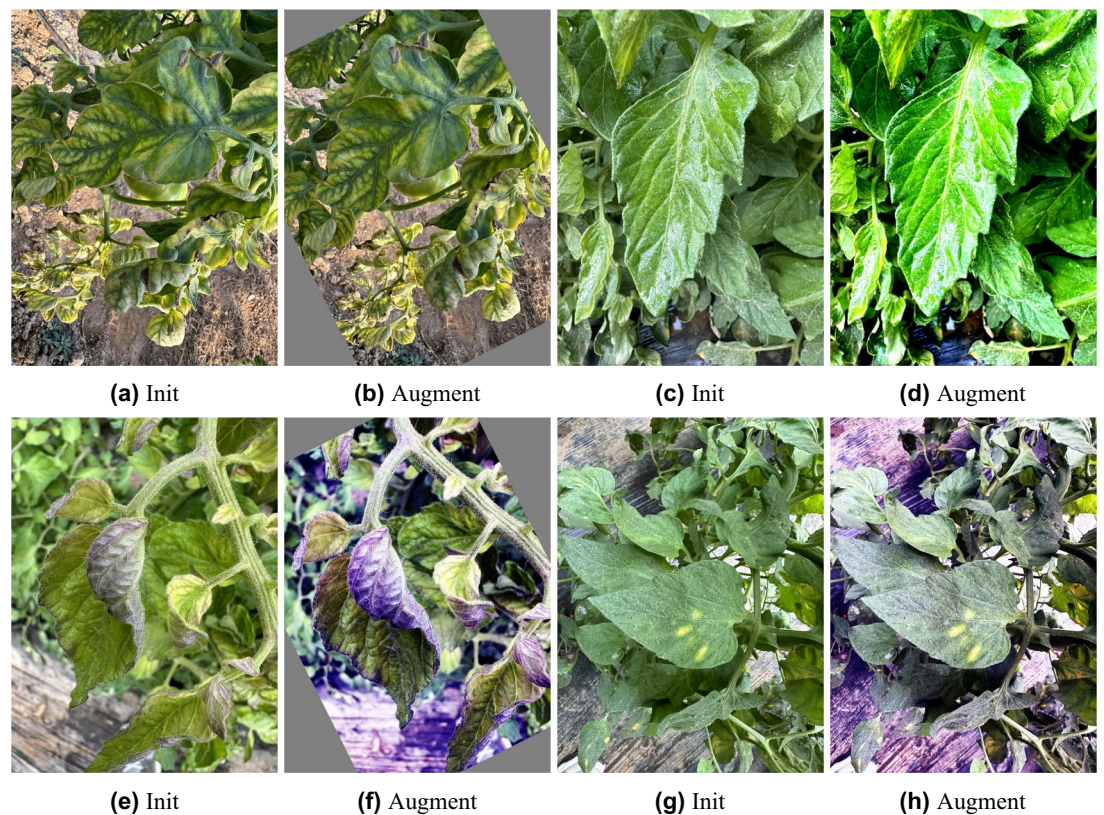


Fig. 4. Image augmentation example.

SE attention mechanism to enhance the model's feature extraction capability. It also uses a branch classifier to improve the model's convergence ability and speed.

In terms of deep feature reuse, the MyDenseNet121 branch is an improvement on DenseNet121. Within the dense block, each layer is directly connected to all previous layers within the block, allowing low-level features extracted by shallow layers to be directly passed to deep layers. This structure effectively alleviates the vanishing gradient problem and improves information flow. However, the overly dense feature fusion method leads to a sharp increase in model parameters. Therefore, this study improved the initial dense block to obtain the N-dense block, which significantly reduces the number of parameters without affecting performance. In terms of multi-receptive field feature extraction, the MyGoogLeNet branch is proposed based on the classic GoogLeNet. This branch consists of four inception blocks, and the module achieves multi-scale feature extraction by using different-sized convolution kernels in parallel, enabling the model to capture feature information within different receptive field ranges simultaneously. Thus, the features of tomato leaf diseases under different receptive fields can be fully extracted.

By concatenating the feature maps extracted by the N-dense block in the MyDenseNet121 branch and the inception block in the MyGoogLeNet branch along the channel dimension, the deep feature reuse capability of DenseNet121 and the multi-scale feature extraction capability of GoogLeNet are effectively combined. Additionally, considering that the channel dimension of the fused feature map is large, the SE attention mechanism is used to focus on the channels containing important information. The Transition Layer is used to reduce the dimension of the output feature map of the N-dense block, thereby avoiding the problem of excessive parameters caused by the large channel dimension of the feature map after subsequent feature fusion. At the tail of DPEN, a branch classifier is used to provide additional gradient signals for the MyDenseNet121 branch model, thereby improving the convergence speed and ability of the model.

The DPEN model has three innovations at the design level: First, it has a parallel architecture that fuses multi-scale and deep reuse, by combining the MyGoogLeNet branch and the MyDenseNet121 branch, the multi-scale feature information extraction and cross-layer feature information reuse complementary are realized, and the model recognition accuracy is improved. Second, it addresses the problem of a sharp increase in channels and parameters caused by dense connections in DenseNet121 by proposing the N-dense block, which improve the feature fusion was improved from layer-by-layer fusion to every three-layer fusion, significantly reducing the number of parameters while retaining the core advantage of feature reuse without significantly sacrificing model recognition performance. Third, it introduces a branch classifier at the model tail to provide additional gradient signals for the MyDenseNet121 branch model, thereby improving the convergence speed and ability of the model.

The structure of the DPEN model is shown in Figure 5. Taking a three-channel image of size $3 \times 224 \times 224$ as input, both branches first perform a 7×7 convolution to extract global features, and then undergo 3×3 max pooling for downsampling. The left MyDenseNet121 branch, based on the principle of feature reuse, uses 4 N-dense blocks to extract disease features, with output channel numbers of 208, 392, 772, and 770 respectively. After the first three N-dense blocks, a transition layer is used to compress the feature map dimensions. Inside the transition layer, 1×1 convolution is used to reduce the channel dimension, and a pooling downsampling with a stride of 2 is used to reduce the feature map size. The right MyGoogLeNet branch combines multi-scale feature extraction with channel attention mechanisms. After initialization, it undergoes 1×1 convolution, 3×3 convolution, and 3×3 pooling processing. Subsequently, 4 inception blocks are used to extract disease feature information under different receptive fields, with output channel numbers of 256, 480, 512, and 832 respectively. An SE Attention module is embedded after each inception block to achieve adaptive weighting of the channel features of the two branches. Feature fusion is accomplished through phased channel concatenation, that is, the outputs of the first three transition layers of the MyDenseNet121 branch and the outputs of the first three inception blocks of the MyGoogLeNet branch are concatenated in sequence, and the outputs of the fourth N-dense block and the fourth inception block complete the final feature fusion. Finally, after the spatial dimensions of the two branches are compressed by global average pooling, they are input into the fully connected layer, and the final prediction results of 4 classifications are output.

N-Dense block

Each layer in the dense block can receive the output of all previous layers, thus having better expressive power and often providing better performance in tasks such as image recognition and object detection. However, owing to its overly dense feature fusion method, the number of parameters within this module increases sharply. Considering that the output feature map needs to be fused with the output of the inception block later, the classic dense block is improved to reduce the number of parameters. First, the dense feature fusion method layer by layer in the classic dense block is improved to a feature fusion method aggregating every three layers. The channel growth rate in the dense block is subsequently reduced from 32 to 24, thereby achieving the goal of reducing the number of parameters in each dense block. The improved dense block module is named the N-dense block. Experiments show that the N-dense block obtained by improving the dense block in this way reduces the number of parameters by 64% compared with the classic dense block without significantly reducing the recognition accuracy of the model. The structure of the dense block is shown in Fig. 6a, and the structure of the N-dense block is shown in Fig. 6b.

Inception block

In the inception block, a multi-branch convolutional architecture is employed to improve the model's ability to represent lesion patterns at different scales. Specifically, 1×1 convolutions are used for linear projection and channel reduction, which reduces the computational cost of the subsequent 3×3 and 5×5 convolutions while preserving salient information. The 3×3 and 5×5 convolutional kernels are dedicated to capturing fine-grained local texture and larger-scale structural information, respectively, thereby accommodating scale variations of lesions ranging from small spots to extensive regions. Simultaneously, a 3×3 max-pooling branch extracts locally salient features and further enhances translation invariance. The multi-scale feature maps produced by these branches are concatenated along the channel dimension to form a fused representation with richer semantic hierarchy and multi-receptive-field characteristics, enabling comprehensive modeling of lesion texture, color variation, and contour structure. The structure of the inception block is shown in Fig. 7.

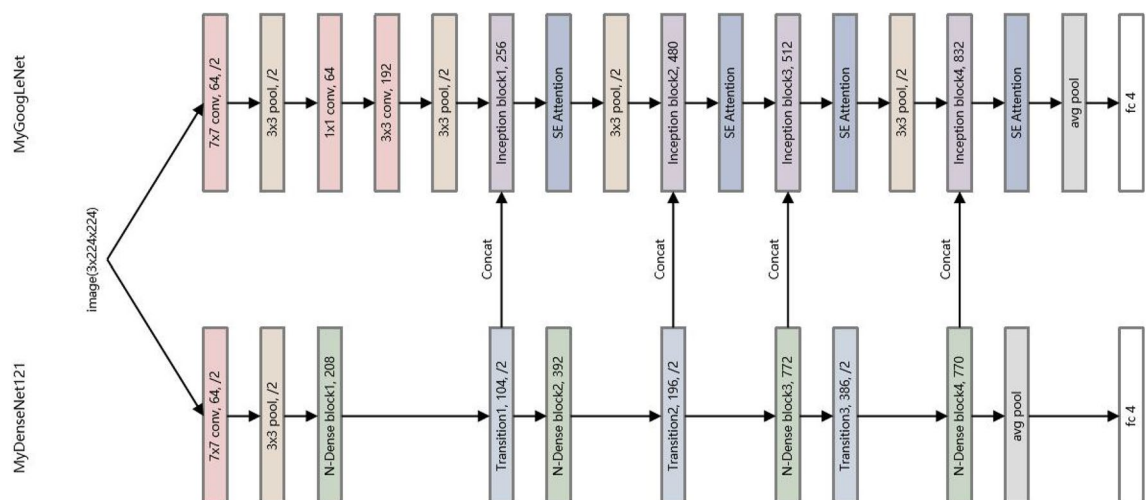


Fig. 5. DPEN model structure.

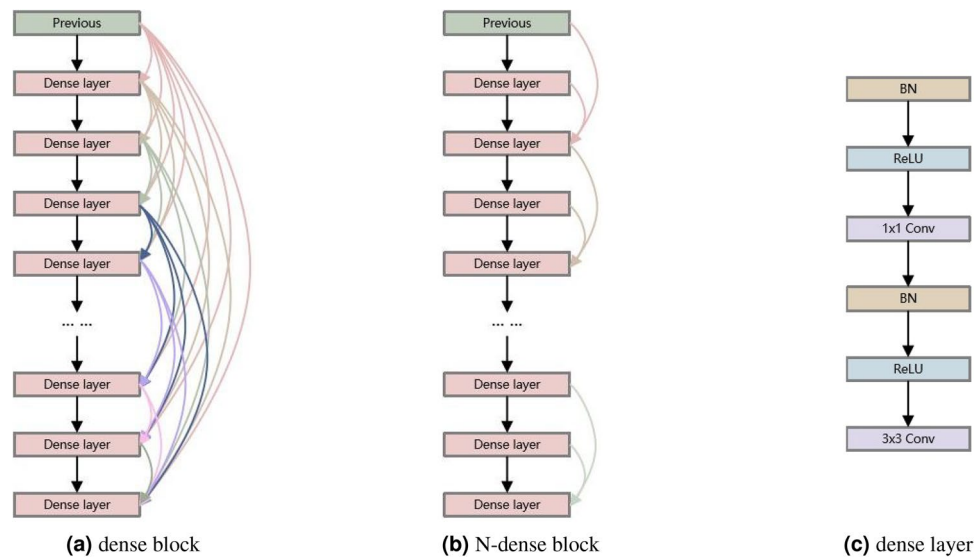


Fig. 6. Examples of dense block (a), N-dense block (b) and a dense layer (c).

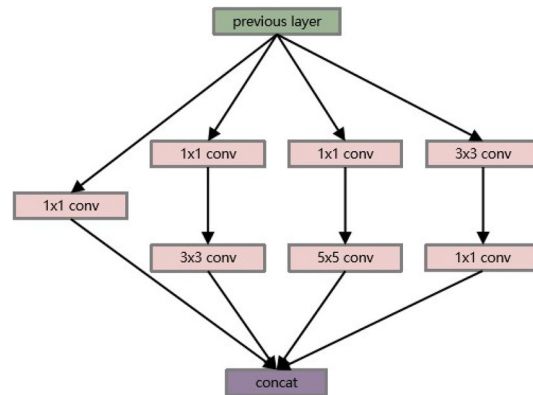


Fig. 7. Architecture of the Inception block.

SE attention mechanism

The squeeze-and-excitation (SE) attention module³⁰ is a channel attention mechanism designed to enhance CNN representations by explicitly modeling dependencies between channels. It learns the importance weights of each channel and adaptively recalibrates channel responses to improve feature selection and the expressive capacity of the model. Specifically, the SE module applies global average pooling to the input feature map, compressing spatial information into a scalar descriptor for each channel. This operation effectively captures global channel context while reducing computational complexity, thereby enabling the model to focus on the most salient features, as illustrated in (1).

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_{ijc} \quad (1)$$

Among them, z_c represents the global description of the c -th channel, and x_{ijc} is the value of the input feature map at position (i, j) . H and W are the height and width of the feature map, respectively.

Two fully connected layers are subsequently used to perform nonlinear transformation on the extracted channel feature scalars to learn the importance weights of each channel, as shown in (2). The learned weights are then mapped to the range of 0 to 1 via the sigmoid activation function to adjust the weights of each channel in the subsequent process, enabling the model to focus more effectively on important features while suppressing redundant information. The sigmoid function is shown in (3).

$$s_c = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot z_c + b_1) + b_2) \quad (2)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

where σ is the sigmoid function, W_1 and W_2 are the weights of the fully connected layers, b_1 and b_2 are the bias terms, and s_c is the weight of the c -th channel.

Finally, the calculated channel weights are multiplied with each channel of the original feature map in a channel wise manner, thereby adjusting the response of each channel, as shown in (4).

$$x'_c = s_c \cdot x_c \quad (4)$$

where s_c represents the c -th channel of the input feature map and where x'_c is the feature map after weighted adjustment.

Considering that the output feature maps of the N-dense block and the inception block are fused by concatenating along the channel dimension, resulting in many channels, the SE attention module is used to assign different weights to each channel after feature fusion, this enables the feature information in the channels containing important information to be highlighted. The structure of the SE attention module is shown in Fig. 8.

Branch classifier

In deep CNN models, gradient values tend to diminish layer by layer during backpropagation, which leads to slow parameter updates and learning difficulties in the lower layers. By adding an extra supervisory signal to the model, additional gradient flow can be provided, enabling better training of the earlier layers and alleviating the problem of gradient vanishing. Additionally, it helps the model learn effective features more quickly, thereby accelerating the overall training process.

In the DPEN model, at the end of the MyDenseNet121 branch model, a global average pooling layer is first used to compress the spatial information of each channel into a single scalar, followed by a fully connected layer for classification prediction, thereby providing an additional supervisory signal for the model training stage. Since the DPEN model has two branch outputs, the outputs need to be weighted and summed. Both the MyDenseNet121 branch classifier and the MyGoogLeNet branch classifier use the cross-entropy loss function, as shown in (5). Owing to the model design that fuses the intermediate layer output of the MyDenseNet121 branch into the MyGoogLeNet branch, the MyDenseNet121 branch classifier only generates classification output during the model training stage, whereas during testing, the output function of this branch classifier is disabled to reduce unnecessary computational processes.

$$\text{Loss} = [\sum_{i=1}^k y_i \log(p_i)] \quad (5)$$

Here, k represents the number of categories, y_i represents the true label, and for $i = k$, $y_i = 1$. p_i is the probability of each category being predicted by the model.

By continuously adjusting the output weight combination of the two branches under the same experimental environment and observing the convergence of the loss function value within 5 training epochs and the improvement in accuracy on the test set, it was ultimately determined that the model converges the fastest when the output weights of the MyDenseNet121 branch classifier and the MyGoogLeNet branch classifier are both 0.5, as shown in (6).

$$\text{Loss}_{\text{sum}} = 0.5 \cdot \text{Loss}_{\text{GoogLe}} + 0.5 \cdot \text{Loss}_{\text{Dense}} \quad (6)$$

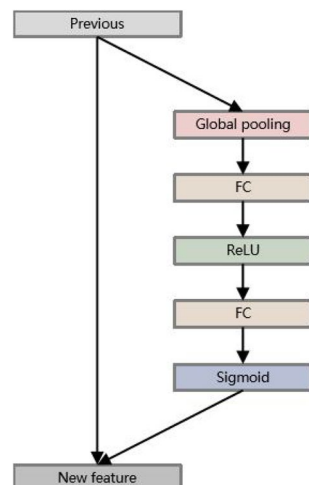


Fig. 8. SE attention module.

$Loss_{sum}$ represents the value of the loss function of the DPEN model. $Loss_{GoogLe}$ represents the value of the loss function of the MyGoogLeNet branch model. $Loss_{Dense}$ represents the value of the loss function of the MyDenseNet121 branch model.

Experimental analysis

Experimental platform and parameter settings

The experiment utilized a remote server, with Python 3.8 as the programming language, PyTorch as the deep learning framework, an RTX 2080 Ti (11 GB) GPU, and CUDA for efficient computing. The CPU used was an Intel(R) Xeon(R) Platinum 8255C. During the training process of the deep learning model, the selection of hyperparameters has a crucial effect on the model's performance. Among them, the learning rate directly determines the convergence speed and the final optimization effect of the model. To ensure the best convergence, this study compared three different learning rates of 0.01, 0.001, and 0.0001 in the experiment to analyze the influence of different learning rates on model training. The experimental results indicated that when the learning rate was set to 0.0001, the model could converge at a faster speed while maintaining a better optimization effect. Therefore, in the subsequent training process, the learning rate was fixed at 0.0001. Additionally, to strike a balance between computing resources and memory consumption, the batch size was set to 32, and the total number of training iterations (epochs) was 50. Moreover, compared with the traditional Stochastic Gradient Descent (SGD) method, the Adaptive Moment Estimation (Adam) optimizer³¹ can dynamically adjust the learning rate, so that the model can approach the optimal solution more quickly. Thus, the training efficiency and the performance of the final model can be improved, so Adam optimizer is used in this experiment.

Evaluation

In this study, we evaluated the tomato leaf disease recognition model in terms of precision, recall rate, F1-Score, and number of parameters. The explanations for each parameter are as follows. The prediction results of the model can be divided into four cases. TP represents the number of samples whose actual category is positive, and the model correctly predicts them as positive. FP represents the number of samples whose actual category is negative, but the model incorrectly predicts them as positive. TN represents the number of samples whose actual category is negative, and the model correctly predicts them as negative. FN represents the number of samples whose actual category is positive but the model wrongly predicts as negative.

- (1) Precision represents the proportion of true positive samples among the samples predicted as positive by the model and is used to measure the precision of the model when predicting the positive category, as shown in (7).

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7)$$

- (2) Recall represents the proportion of positive samples correctly predicted by the model. Compared with all actual positive samples, it is used to measure the model's ability to identify positive samples, as shown in (8).

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

- (3) The F1-score is the weighted harmonic mean of precision and recall, as shown in (9).

$$F_1 - \text{Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

- (4) The number of parameters refers to the total number of all trainable parameters in a model. It is an important indicator for measuring the complexity of a model and is usually directly related to the model's computational and storage requirements.

Results and analysis

The confusion matrix, also known as the error matrix, is an important tool for evaluating the performance of classification models. It visually presents the correspondence between the model's prediction results and the true labels in matrix form. The rows of the confusion matrix represent the actual categories, and the columns represent the categories predicted by the model. The values on the main diagonal indicate the number of samples of each category correctly classified by the model, reflecting the classification performance of the model in each category. By analyzing the confusion matrix, researchers can intuitively observe which categories the classification model performs well in and which categories are prone to misclassification. Additionally, the confusion matrix can be used to calculate accuracy, precision, recall, and F1-Score metrics, thereby helping researchers comprehensively evaluate model performance.

In this study, five advanced models, namely, GoogLeNet¹¹, DenseNet121¹³, AlexNet⁹, ResNet18¹², and EfficientNet³², were selected, along with the DPEN model constructed in this paper, to conduct experimental comparisons on the self-built dataset of tomato leaf diseases in complex backgrounds. All the models were trained from scratch under the same experimental environment and hyperparameter conditions to ensure fairness and comparability. According to the confusion matrix, the DPEN has the best recognition effect on the four types of

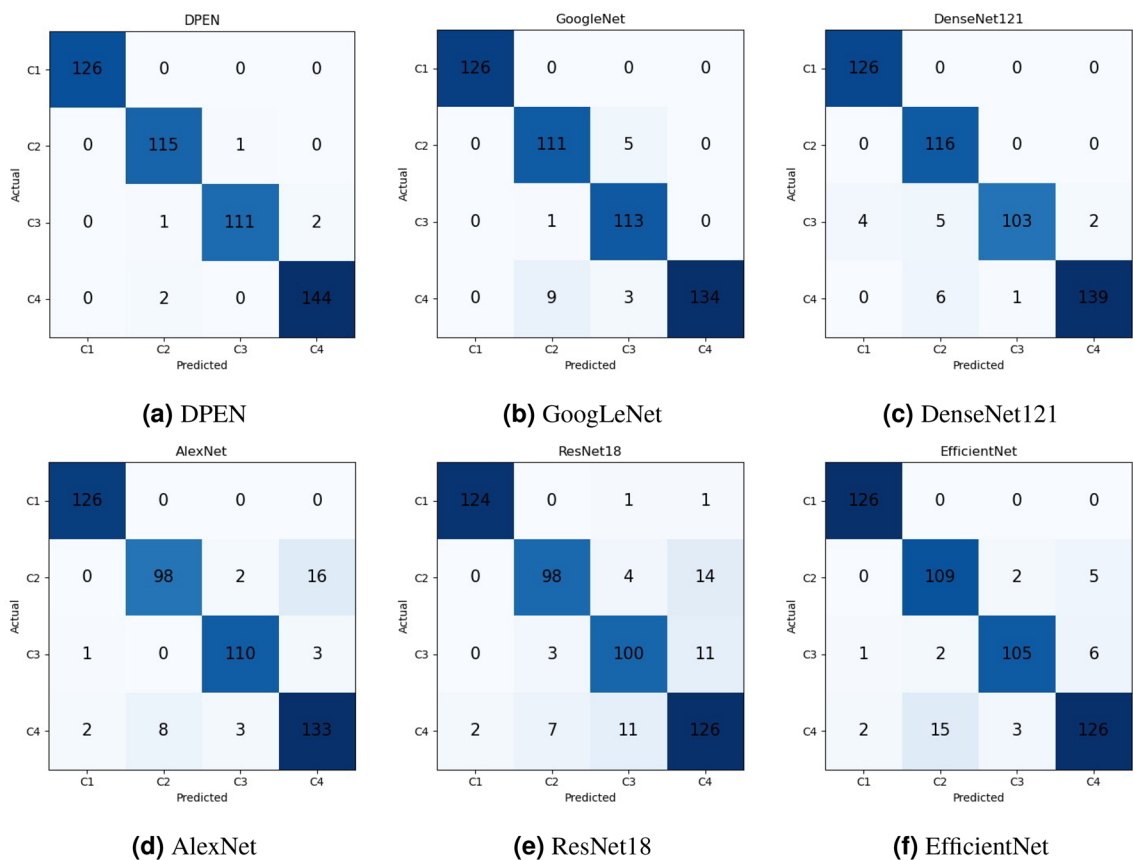


Fig. 9. Confusion matrices (C1, C2, C3 and C4 denote Mg deficiency, healthy, virus disease and leaf mold, respectively).

Models	Precision/%	Recall/%	F1-Score/%	Parameters/M
DPEN	98.80	98.79	98.79	5.91
GoogLeNet	96.28	96.65	96.46	13.00
DenseNet121	96.47	96.39	96.43	7.98
AlexNet	93.32	93.02	93.17	61.10
ResNet18	89.56	89.23	89.39	11.69
EfficientNet	92.90	93.10	93.00	5.29

Table 2. Experimental results of the self-built dataset.

complex background tomato leaf images in the self-built dataset, with the fewest samples misclassified in each category. The confusion matrices of the six models are shown in Fig. 9.

The recognition performance of different models on the self-built dataset is shown in Table 2. The experimental results indicate that DPEN performs best in the task of tomato leaf disease recognition, with precision, recall, and F1-Score reaching 98.80%, 98.79%, and 98.79% respectively, significantly outperforming other comparison models. Notably, DPEN has only 5.91 M parameters, suggesting its potential for deployment on platforms with limited computing resources. The outstanding performance of DPEN is mainly attributed to its effective integration of the structural advantages of GoogLeNet and DenseNet121. On the one hand, it inherits the multi-scale feature extraction capability of GoogLeNet, and on the other hand, it integrates the dense connection mechanism of DenseNet121, thereby effectively improving the accuracy of tomato leaf disease recognition.

In contrast, the precision, recall, and F1-Score of GoogLeNet are 96.28%, 96.65%, and 96.46% respectively, those of DenseNet121 are 96.47%, 96.39%, and 96.43% respectively. The overall recognition performance of the two models in this task is similar, indicating that despite the differences in their structural design concepts, both can achieve high classification accuracy in leaf disease recognition. The performance of AlexNet and ResNet18 is relatively poor, with F1-Scores of 93.17% and 89.39% respectively. The main reason for this difference is that the shallow network architecture of AlexNet limits its ability to capture fine-grained features of leaf spots. Although ResNet18 improves training stability through residual structures, its network depth and capacity are still insufficient to fully extract the multi-level semantic information of complex images in this task. As a lightweight

model, EfficientNet has an advantage in terms of parameter quantity, with only 5.29 M parameters, but its F1-Score is 93.00%, showing a performance gap of 5.79% compared to DPEN.

The recognition results for the four categories of diseases on the self-built dataset are presented in Table 3. Significant differences are observed among the models in terms of precision, recall, and F1-Score. Overall, DPEN demonstrates high stability across all four categories. For C1, all three metrics achieve 100.00%, while for C2, C3, and C4, the precision values are 97.46%, 99.11%, and 98.63%, and the recall values are 99.14%, 97.37%, and 98.63%, respectively. The F1-Scores for all four categories are the highest among the compared models. These results indicate that DPEN maintains consistent discriminative capability across different disease types, reflecting that its integration of inception blocks and dense blocks effectively adapts to the visual variations of various diseases.

GoogLeNet showed significant performance differences in four types of leaf disease identification tasks. In category C1, both precision and recall reached 100%, indicating that DenseNet121 could perfectly distinguish the leaves with this type of disease. In category C2, precision and recall were 91.74% and 95.69% respectively, suggesting that the recognition effect was relatively stable but had room for improvement. The precision in categories C3 and C4 was 93.39% and 100% respectively, while the recall was 99.12% and 91.78% respectively. There was a significant difference in performance, indicating a considerable number of false detections in category C3 and a relatively severe number of missed detections in category C4.

The precision and recall of DenseNet121 on C1 are 96.92% and 100% respectively, indicating that the recognition effect on this category is relatively stable, but there is still room for improvement in precision. For the C2 category, the precision is 91.34% while the recall reaches 100.00%, suggesting that there is considerable misclassification in the detection of this category. The precision and recall for C3 are 99.04% and 90.35% respectively, with a high precision but a low recall, indicating a relatively severe under-detection issue in this category. On the C4 category, the precision is 98.58% and the recall is 95.21%, suggesting that the recognition effect is also relatively stable but has some room for improvement.

It is worth noting that GoogLeNet and DenseNet121 show complementarity. Specifically, in category C1, GoogLeNet has a precision of 100%, while DenseNet121's precision is only 96.92%. In category C2, GoogLeNet has a recall rate of 95.69%, while DenseNet121's recall rate is 100%. In category C3, GoogLeNet has a relatively high recall rate but a low precision, while DenseNet121 complements it by having a high precision but a low recall rate. In category C4, GoogLeNet's recall rate is only 91.78%, while DenseNet121's recall rate is 95.21%. Further analysis of DPEN reveals that its F1-Score in all four categories has reached the best value, which further proves that DPEN effectively combines GoogLeNet and DenseNet121, thereby fully leveraging their respective advantages and achieving higher recognition stability in each category.

The recognition performance of AlexNet in the four disease categories is relatively weaker compared to the above three models. Specifically, for the C1 category, the precision, recall, and F1-Score are 97.67%, 100.00%, and 98.82% respectively, for the C2 category, the precision is 92.45%, the recall is 84.48%, and the F1-Score is 88.29%, for the C3 category, the precision, recall, and F1-Score are 95.65%, 96.49%, and 96.07% respectively, for the C4 category, they are 87.50%, 91.10%, and 89.26% respectively. It can be seen that AlexNet has poor recognition effect on C2 and C4 categories. From the perspective of model design, as one of the classical convolutional neural networks, AlexNet has a relatively small number of model layers, and the number of convolution and pooling layers is also limited. Therefore, it is difficult to effectively capture the multi-scale texture, shape, and fine changes of diseases, which leads to its recognition performance degradation.

Model	Metric	C1	C2	C3	C4
DPEN	Precision/%	100.00	97.46	99.11	98.63
	Recall/%	100.00	99.14	97.37	98.63
	F1-Score/%	100.00	98.29	98.23	98.63
GoogLeNet	Precision/%	100.00	91.74	93.39	100.00
	Recall/%	100.00	95.69	99.12	91.78
	F1-Score/%	100.00	93.67	96.17	95.71
DenseNet121	Precision/%	96.92	91.34	99.04	98.58
	Recall/%	100.00	100.00	90.35	95.21
	F1-Score/%	98.44	95.47	94.50	96.87
AlexNet	Precision/%	97.67	92.45	95.65	87.50
	Recall/%	100.00	84.48	96.49	91.10
	F1-Score/%	98.82	88.29	96.07	89.26
ResNet18	Precision/%	98.41	90.74	86.21	82.89
	Recall/%	98.41	84.48	87.72	86.30
	F1-Score/%	98.41	87.50	86.96	84.56
EfficientNet	Precision/%	97.67	86.51	95.45	91.97
	Recall/%	100.00	93.97	92.11	86.30
	F1-Score/%	98.82	90.09	93.75	89.04

Table 3. Performance of different models on each category of the self-built dataset (C1, C2, C3 and C4 denote Mg deficiency, healthy, virus disease and leaf mold, respectively).

The recognition performance of ResNet18 is slightly lower than that of AlexNet. For category C1, its precision, recall, and F1-Score are all 98.41%, for C2, the precision is 90.74%, the recall is 84.48%, and the F1-Score is 87.50%, for C3 category, the three indicators are 86.21%, 87.72%, and 86.96%, for C4 category, they are 82.89%, 86.30%, and 84.56%. It can be seen that the recognition effect of this model on C2, C3 and C4 is poor. Also as the most classical CNN model, although ResNet18 effectively alleviates the problem of gradient vanishing through the residual structure and improves the efficiency of feature transmission, due to its relatively shallow network depth and limited number of parameters, its representational ability is still mainly concentrated on mid-low-level semantic features. There are deficiencies in the differentiated expression of multi-scale structural textures and fine lesion areas. Especially in the categories with similar lesion differences, the model is difficult to construct sufficient rich high-level semantic features, resulting in a decline in classification discrimination ability.

The performance of EfficientNet varies across different categories. For the C1 category, the precision, recall rate, and F1-Score are 97.67%, 100.00%, and 98.82% respectively, for the C2 category, they are 86.51%, 93.97%, and 90.09%, for the C3 category, they are 95.45%, 92.11%, and 93.75%, and for the C4 category, they are 91.97%, 86.30%, and 89.04%. The recognition effect of the proposed model is slightly inferior on the two categories C2 and C4. Although EfficientNet optimizes the width, depth and input resolution of the network through a composite scaling strategy, enabling it to efficiently extract features with fewer parameters, it is difficult for the model to stably distinguish subtle pathological differences in categories with similar disease morphology or small symptom regions, such as C2 and C4 categories. This results in fluctuations in precision and recall between different classes.

Furthermore, by comparing the F1-Scores of AlexNet, ResNet18 and Efficientnet on categories C2 and C4, it is clearly evident that the recognition performance of these two categories is significantly lower than that of the other categories. To further verify the confusion phenomenon between C2 and C4, we conducted a qualitative and quantitative comparison by combining the sample images in Fig. 1 with the confusion matrix in Fig. 9. Qualitatively, the lesions in C4 are smaller and more localized, accounting for a lower proportion in the leaves, and visually they show little difference from the performance of C2. Quantitatively, the confusion matrices of these three models show that a considerable portion of the samples predicted as C2 are mistakenly classified as C4. Similarly, in the samples predicted as C4, a considerable portion actually belong to C2, presenting a clear bidirectional misclassification phenomenon. This phenomenon indicates two points, Firstly, the low-order disease features extracted by shallow networks are insufficient to distinguish the distribution differences of some categories. Secondly, the visual features of the C2 category and the C4 category are relatively similar, making it difficult for the classifier to form a stable decision boundary.

In summary, DPEN maintains high levels of precision, recall, and F1-Score with minimal fluctuation across all four disease categories, outperforming the other five models. These results validate the effectiveness of DPEN's multi-scale feature fusion and dense connectivity strategy in tomato leaf disease recognition, enabling robust classification performance across diverse disease types.

To further verify the reliability of the DPEN model designed in this paper, experiments were conducted on the tomato-leaf-illness-detection public dataset. The experimental results are shown in Table 4. Compared with that of the control model, the precision of the DPEN model increases by 0.33% to 11.64%, and it achieves the best performance in terms of recall and F1-Score, further demonstrating its superiority. Compared with the experimental results on the self-built dataset, the precision, recall, and F1-Score of the DPEN on the public dataset are lower. We believe this result is reasonable. The main reason is that the number of samples in each category of the training set of the public dataset is less than 100. Although the sample size of each category in the training set was expanded to 1000 through data augmentation technology, the model still has difficulty fully learning the disease characteristics within the limited sample space. In addition, the recognition precision of ResNet18 and EfficientNet decreases sharply when the training set sample size is small, indicating that these two models cannot effectively learn the characteristics of various diseases from a small amount of data.

Ablation experiment

In order to verify whether N-dense blocks will affect the recognition precision of DenseNet121 and the effectiveness of SE attention module in DPEN model, we conduct ablation experiments on self-built datasets. In Table 5. “Base” represents the basic DenseNet121 model, “Base_N-Dense” represents the model where the dense block in the base model is replaced with the N-dense block, “DPEN_No_SE” refers to the DPEN model without the SE attention mechanism, and “DPEN” represents the model that uses the SE attention mechanism.

The experimental results indicate that when the dense block in DenseNet121 is replaced with the N-dense block, the model's recognition precision is 94.53%, which is 1.94% lower than that of the DenseNet121 model,

Models	Precision/%	Recall/%	F1-Score/%	Parameters/M
DPEN	96.05	96.55	96.30	5.91
GoogLeNet	94.69	94.34	94.52	13.00
DenseNet121	95.72	95.51	95.61	7.98
AlexNet	92.39	91.42	91.90	61.10
ResNet18	84.41	83.98	84.20	11.69
EfficientNet	84.95	85.33	85.14	5.29

Table 4. Experimental results on a public dataset of tomato leaf illness detection.

Model	Precision/%	Recall/%	F1-Score/%	Parameters/M
Base	96.47	96.39	96.43	7.98
Base_N-Dense	94.53	94.41	94.47	2.84
DPEN_No_SE	97.73	97.85	97.79	5.31
DPEN	98.80	98.79	98.79	5.91

Table 5. Results of ablation experiments.

but the number of parameters is reduced by 5.14 M. This indicates that merely improving the feature fusion method within the dense block does not have a significant effect on the model's performance and can effectively reduce the model's parameter count. When the SE attention mechanism is used, the precision of the DPEN is 98.80%, which is 1.07% higher than the 97.73% recognition precision of the model without the SE attention module. This shows that the SE attention mechanism can enhance the model's focus on channels containing important feature information, thereby further improving the model's recognition accuracy. This result validates the effectiveness of the N-dense block and the SE attention mechanism, thereby effectively enhancing the model's recognition accuracy.

Conclusion

In this study, we proposed the DPEN model, which effectively integrates multi-scale feature extraction and dense feature reuse through a dual-parallel architecture. Experimental results on a self-built dataset of complex background tomato leaf images demonstrated that DPEN significantly enhanced the ability of disease feature learning and disease recognition. Specifically, compared with GoogLeNet and DenseNet121, the precision of DPEN increased by 2.52% and 2.33% respectively, while reducing the parameter count by 7.09 M and 2.07 M, achieving a good balance between precision and model compactness. Experiments on public datasets further confirmed that the proposed model can accurately identify tomato leaf diseases in complex backgrounds.

In addition, DPEN demonstrates strong ability in distinguishing visually similar diseases, being able to precisely distinguish two highly similar leaf categories, C2 and C4, in the self-built dataset, even in the presence of severe background interference, and accurately capturing subtle lesion patterns. These results validate the effectiveness of the proposed dual-parallel feature fusion strategy in complex agricultural environments.

Overall, DPEN provides a solution for real-time plant disease monitoring and intelligent crop protection, suitable for scenarios of tomato leaf disease recognition in images with complex backgrounds.

Despite the satisfactory performance of DPEN, several challenges remain. For instance, its ability to handle multiple disease coexistence and severe leaf occlusion requires further improvement. Additionally, although the number of parameters has been reduced, model deployment on low-power mobile devices may still face limitations. Future work will focus on the following aspects: (1) Build a larger and more diverse dataset covering different planting environments, crop varieties and disease types to enhance the model's ability to identify various diseases of multiple crops; (2) Explore lightweight architectures and model compression techniques to achieve efficient deployment on devices with extremely limited computing resources; (3) Integrate multimodal information to improve the model's recognition accuracy when dealing with complex visual conditions through the collaboration of multiple modal data.

Data availability

The datasets generated during and/or analysed during the current study are available from the corresponding author on reasonable request.

Received: 22 May 2025; Accepted: 30 March 2026

Published online: 12 April 2026

References

- Li, X., Hu, X., Li, F. & Xu, J. Lightweight recognition for multiple and indistinguishable diseases of apple tree leaf. *Trans. Chin. Soc. Agric. Eng.* **39**, 184–190 (2023).
- Guo, X., Fan, T. & Shu, X. Tomato leaf diseases recognition based on improved multi-scale alexnet. *Trans. Chin. Soc. Agric. Eng.* **35**, 162–169. <https://doi.org/10.11975/j.issn.1002-6819.2019.13.018> (2019).
- Attallah, O. Tomato leaf disease classification via compact convolutional neural networks with transfer learning and feature selection. *Horticulturae* **9**, 149. <https://doi.org/10.3390/horticulturae9020149> (2023).
- Kumari, S., Kumari, N. & Nuparam. Plant leaf disease identification using machine learning. In *2022 11th International Conference on System Modeling and Advancement in Research Trends (SMART)*, 1466–1476, <https://doi.org/10.1109/SMART55829.2022.10047040> (2022).
- Fang, C. & Shi, F. Image recognition of tomato diseases based on improved deep residual network. *J. Comput. Appl.* **40**, 203–208. <https://doi.org/10.11772/j.issn.1001-9081.2019081493> (2020).
- Su, S., Qiao, Y. & Rao, Y. Recognition of grape leaf diseases and mobile application based on transfer learning. *Trans. Chin. Soc. Agric. Eng.* **37**, 127–134. <https://doi.org/10.3969/j.issn.1002-6819.2021.10.017> (2021).
- Zhao, X., Li, K., Li, Y., Ma, J. & Zhang, L. Identification method of vegetable diseases based on transfer learning and attention mechanism. *Computers and Electronics in Agriculture* **193**, <https://doi.org/10.1016/j.compag.2022.106703> (2022).
- LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **86**, 2278–2324. <https://doi.org/10.1109/5.726791> (1998).
- Krizhevsky, A., Sutskever, I. & Hinton, G. E. Imagenet classification with deep convolutional neural networks. *Commun. ACM* **60**, 84–90. <https://doi.org/10.1145/3065386> (2017).

10. Simonyan, K. & Zisserman, A. Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations (ICLR)* (2015).
11. Szegedy, C. et al. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1–9, <https://doi.org/10.1109/CVPR.2015.7298594> (2015).
12. He, K., Zhang, X., Ren, S. & Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778, <https://doi.org/10.1109/CVPR.2016.90> (2016).
13. Huang, G., Liu, Z., Van Der Maaten, L. & Weinberger, K. Q. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2261–2269, <https://doi.org/10.1109/CVPR.2017.243> (2017).
14. Liu, Y. et al. Tomato leaf disease recognition based on an optimized swin transformer. *Journal of China Agricultural University* **28**, 80–90. <https://doi.org/10.11841/j.issn.1007-4333.2023.04.07> (2023).
15. Guo, H. et al. Recognition and application of apple defoliation disease based on transfer learning. *Transactions of the Chinese Society of Agricultural Engineering* **40**, 184–192. <https://doi.org/10.11975/j.issn.1002-6819.202308040> (2024).
16. Liu, B., Zhang, Y., He, D. & Li, Y. Identification of apple leaf diseases based on deep convolutional neural networks. *Symmetry* **10**, 11. <https://doi.org/10.3390/sym10010011> (2018).
17. Xu, L., Cao, B., Ning, S., Zhang, W. & Zhao, F. Peanut leaf disease identification with deep learning algorithms. *Mol. Breed.* **43**, 25. <https://doi.org/10.1007/s11032-023-01370-8> (2023).
18. Dhanalakshmi, R., K. B., Sinha, B. B. & Gopalakrishnan, R. Tomato leaf disease identification by modified inception based sequential convolution neural networks. *The Imaging Science Journal* **71**, 408–424, <https://doi.org/10.1080/13682199.2023.2183318> (2023).
19. Xu, Y., Kong, S., Chen, Q., Gao, Z. & Li, C. Model for identifying strong generalization apple leaf disease using transformer. *Nongye Gongcheng Xuebao/Transactions of the Chinese Society of Agricultural Engineering* **38**, 198–206. <https://doi.org/10.11975/j.issn.1002-6819.2022.16.022> (2022).
20. Hidayatulloh, A., Nursalman, M. & Nugraha, E. Identification of tomato plant diseases by leaf image using squeezeNet model. In *2018 International Conference on Information Technology Systems and Innovation (ICITSI)*, 199–204, <https://doi.org/10.1109/ICITSI.2018.8696087> (2018).
21. Wang, R., Chen, F., Zhu, X. & Zhang, X. Identifying apple leaf diseases using improved efficientNet. *Nongye Gongcheng Xuebao/Transactions of the Chinese Society of Agricultural Engineering* **39**, 201–210. <https://doi.org/10.11975/j.issn.1002-6819.202306140> (2023).
22. Huang, X. et al. Tomato leaf disease detection system based on fc-sndpn. *Multimed. Tools Appl.* **82**, 2121–2144. <https://doi.org/10.1007/s11042-021-11790-3> (2023).
23. Liu, C., Mo, H. & Ma, C. Plant disease identification method in real scenarios based on transformer and cnn. *Modern Computer* **29**, 22–27. <https://doi.org/10.3969/j.issn.1007-1423.2023.11.004> (2023).
24. Omer, S. M., Ghafoor, K. Z. & Askar, S. K. Lightweight improved yolov5 model for cucumber leaf disease and pest detection based on deep learning. *SIVIP* **18**, 1329–1342. <https://doi.org/10.1007/s11760-023-02865-9> (2024).
25. Zhang, G. et al. Model for identifying lotus leaf pests and diseases using improved densenet and transfer learning. *Nongye Gongcheng Xuebao/Transactions of the Chinese Society of Agricultural Engineering* **39**, 188–196. <https://doi.org/10.11975/j.issn.1002-6819.202301122> (2023).
26. Tewari, A. S. & Kumari, P. Lightweight modified attention based deep learning model for cassava leaf diseases classification. *Multimed. Tools Appl.* **83**, 57983–58007. <https://doi.org/10.1007/s11042-023-17459-3> (2024).
27. Upadhyay, N. & Gupta, N. Seglearner: A segmentation based approach for predicting disease severity in infected leaves. *Multimedia Tools and Applications* 1–24, <https://doi.org/10.1007/s11042-025-20838-7> (2025).
28. Liu, Y. & Gao, G. Identification of multiple leaf diseases using improved squeezeNet model. *Nongye Gongcheng Xuebao/Trans. Chin. Soc. Agric. Eng.* **37**, 187–195. <https://doi.org/10.11975/j.issn.1002-6819.2021.2.022> (2021).
29. Cubuk, E. D., Zoph, B., Mané, D., Vasudevan, V. & Le, Q. V. Autoaugment: Learning augmentation strategies from data. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 113–123, <https://doi.org/10.1109/CVPR.2019.00020> (2019).
30. Hu, J., Shen, L. & Sun, G. Squeeze-and-excitation networks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7132–7141, <https://doi.org/10.1109/CVPR.2018.00745> (2018).
31. Kingma, D. & Ba, J. Adam: A method for stochastic optimization. *Computer Science* <https://doi.org/10.48550/arXiv.1412.6980> (2014).
32. Tan, M. & Le, Q. V. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 6105–6114 (2019).

Author contributions

Xin Zhang wrote the main content of the manuscript; Xin Zhang and Yuxin Tian collected the data and prepared the illustrations for the paper; Wanlin Gao, Yuxin Tian and Yinxue Shi reviewed the manuscript.

Funding

This research was supported by the Development Research of Smart Agriculture Discipline of Pu'er University.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Y.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026